2026 International Conference on Unmanned Aircraft Systems (ICUAS)
June 15-18, 2026. Corfu, Greece

Trajectory Planning for an Omnidirectional Drone in a GPS-Denied and Obstacle Cluttered Environment

Abdullah Mohamed Ali[1], Mahmoud Hamandi[2], Anthony Tzes[1]

Abstract— **Omnidirectional drones offer a unique advantage over classical multirotor UAVs by enabling independent tracking of both three-dimensional position and orientation. This added flexibility makes trajectory planning significantly more challenging than planning for classical drones. In this paper, we present a trajectory planning framework tailored for omnidirectional UAVs operating in unknown, cluttered environments. The vehicle is equipped with a forward-facing RGB-D camera and an IMU, and builds an RTAB-Map with loop closures during flight. Our method incrementally identifies feasible subgoals from the explored free space and generates full-pose safe trajectories—covering both position and orientation—to reach them. This process is repeated until the global target becomes accessible. The proposed approach is validated in a high-fidelity physics simulator, using our omnidirectional drone, within an environment that requires precise navigation through tight passages and complex orientations. The planner's effectiveness is demonstrated in scenarios that are otherwise infeasible for conventional UAVs.**

I. INTRODUCTION

Search-and-rescue missions present unique challenges due to the unpredictable and hazardous nature of disaster environments, where direct human intervention is often unsafe or infeasible. As a result, robotic systems are increasingly being developed to autonomously navigate debris-filled areas, search for survivors, identify viable passages, and deliver critical information or supplies [1]. Beyond disaster response, many applications require UAVs to operate in complex three-dimensional spaces with dense obstacles, such as industrial plants, underground tunnels, and confined infrastructure environments [2]–[4].

Uncrewed Aerial Vehicles (UAVs) offer clear advantages over other robotic platforms thanks to their ability to operate freely within the full three-dimensional aerial workspace [5]. Nevertheless, conventional coplanar-collinear multirotor configurations suffer from inherent coupling between lateral translation and attitude, which restricts their maneuverability. As a result, these platforms cannot perform certain tasks, such as traversing confined passages at steep or vertical angles. One intuitive workaround is to employ smaller UAVs that can fit through narrow openings; however, this approach inherently limits both vehicle size and payload capacity, which may be critical for exploration and sensing.

To overcome these limitations, research has increasingly focused on the development of Fully Actuated UAVs [6], where translational and rotational dynamics are decoupled. Among these, omnidirectional designs [7] provide the ability to hover and maneuver in arbitrary orientations, significantly enhancing agility and adaptability in cluttered environments. By allowing the vehicle to exploit its full actuation authority, such designs expand operational capabilities well beyond those of classical UAV architectures.

Omnidirectional UAVs have recently attracted considerable attention for their ability to maneuver efficiently in constrained environments. Leveraging these advantages, researchers have proposed new designs aimed at improving efficiency and reducing actuator count [8], [9], while others have focused on application-driven designs such as aerial physical interaction [10].

In this work, we focus on the problem of trajectory planning for omnidirectional UAVs operating in dense and cluttered environments. All experiments are conducted with the *OmniOcta*, an omnidirectional platform designed for efficiency without tilting mechanisms, equipped with eight fixed propellers [11], [12]. Our proposed trajectory generation framework is validated in a physics engine using the *OmniOcta*. Figure 1 illustrates the platform and its integrated depth-sensing system, employed in the Simultaneous Localization and Mapping (SLAM).



Fig. 1: OmniOcta: omnidirectional drone equipped with an RGB-D camera used for visual–inertial SLAM.

For underactuated quadrotors, several trajectory planning methods have been developed with a primary emphasis on safe navigation in cluttered environments. The EGO planner [13] formulates the problem as a convex decomposition of free space, enabling real-time computation of collision-free and smooth position trajectories. Building on this, the EVA planner [14] introduces a sequence of convex optimization problems to further improve efficiency and robustness in dynamic scenarios. Polynomial trajectory optimization [15] is another widely used approach, where piecewise polynomial representations are employed to com-

[1] Center for Artificial Intelligence and Robotics, New York University Abu Dhabi, Saadiyat Island, 129188, Abu Dhabi, UAE abdullah.ali@nyu.edu, anthony.tzes@nyu.edu

[2] Mahmoud Hamandi is with Mohamed bin Zayed University of Artificial Intelligence, Masdar City, Abu Dhabi, UAE. This work was conducted while he was working at the Center for Artificial Intelligence and Robotics, New York University Abu Dhabi, Saadiyat Island, 129188, Abu Dhabi mahmoud.hamandi@mbzuai.ac.ae

This work has been supported by the NYUAD Center for Artificial Intelligence and Robotics, funded by Tamkeen under the NYUAD Research Institute Award CG010.

2026 International Conference on Unmanned Aircraft Systems (ICUAS) | 979-8-3315-9316-2/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICUAS69441.2026.11598736

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 03:53:39 UTC from IEEE Xplore. Restrictions apply.

pute minimum-snap solutions that are both computationally efficient and well-matched to quadrotor dynamics. Although highly effective for quadrotor navigation, these methods are limited to translational planning and do not extend naturally to platforms that can exploit decoupled position–orientation trajectories.

Trajectory generation for fully actuated multirotor vehicles has also been investigated, with research often emphasizing computational tractability or energy-aware design. In [16], a fast trajectory computation strategy is presented that guarantees dynamic feasibility, but does not account for obstacles, restricting its applicability to clutter-free scenarios. An energy-efficient optimization framework is introduced in [17] for a hexarotor with additional actuation, successfully reducing energy consumption but without integration of collision avoidance. A different perspective is proposed in [18], where a Riemannian geometry-based formulation ensures smooth and dynamically consistent 6-DoF motions. However, as with the other approaches, the absence of explicit obstacle avoidance remains a critical limitation for operation in real-world environments.

In [19], a framework for computing collision-free 6D trajectories in cluttered environments is presented. Their approach represents free space as a collection of convex polyhedra and then fits a trajectory such that a bounding box enclosing the UAV remains entirely within these safe regions. Although effective, the method assumes known obstacles, represented by bounding boxes, and an initial path computed from start to goal, rendering the method difficult to deploy in a real-world scenario where the environment is discovered during the planning phase.

Recently, our previous work [20], [21] demonstrated the advantage of 6D path planning for an omnidirectional drone, enabling traversal through tight and difficult-to-maneuver openings. In that work, we proposed a split framework in which the path was first computed and then a trajectory was fitted to it. The resulting trajectory adapted its speed by slowing down near obstacles and accelerating in open spaces, achieving a balance between efficiency and safety. Finally, [21] presented the application of the algorithm in real hardware experiments. However, in this work the environment was assumed to be known apriori, making its application limited to discovered environments.

Hence, there remains a need for a trajectory planning approach that fully exploits the ability of omnidirectional drones to follow 6-dimensional trajectories, while navigating an unknown, cluttered environment, with challenging to navigate passages. The contributions of this paper are as follows:

- **Incremental global path planning:** We introduce a frontier-inspired strategy for omnidirectional UAVs operating in unknown and cluttered environments. The method relies on SLAM to both map the environment and localize the vehicle, while incrementally selecting safe sub-goals that drive exploration toward the global objective.
- **Multi-stage local trajectory generation:** We develop a hierarchical planner that first computes a collision-free 3D path using A*, then refines it into a smooth 6D path that couples position and orientation while avoiding obstacles. A polynomial trajectory is finally fitted to the optimized path to ensure dynamic feasibility.
- **Full 6D trajectory exploitation:** Unlike prior works that assume known environments or focus only on translational planning, our approach explicitly leverages the omnidirectional UAV's ability to follow coupled position–orientation trajectories in previously unseen cluttered spaces.
- **Validation:** We validate the proposed framework in Software-in-the-Loop (SITL) simulations within a physics engine modeling the OmniOcta platform [11], demonstrating safe and efficient navigation in complex environments.

II. Mathematical Modeling

A. Dynamical Model

We consider a body-fixed frame $\mathcal{F}_B$ attached to the UAV's Center of Mass (CoM), with axes $\{\boldsymbol{x}_B, \boldsymbol{y}_B, \boldsymbol{z}_B\}$ and origin O_B. The inertial space is described by a fixed frame $\mathcal{F}_W$ with axes $\{\boldsymbol{x}_W, \boldsymbol{y}_W, \boldsymbol{z}_W\}$ and origin O_W. The position (orientation) of $\mathcal{F}_B$ relative to $\mathcal{F}_W$ is denoted by $\boldsymbol{p}_B \in \mathbb{R}^3$ ($\boldsymbol{q}_B \in \mathbb{H}$), while the linear (angular) velocity of O_B in $\mathcal{F}_W$ is given by $\boldsymbol{v}_B \in \mathbb{R}^3$ ($\boldsymbol{\omega}_B \in \mathbb{R}^3$). Similarly, the linear (angular) acceleration of O_B in $\mathcal{F}_W$ is denoted by $\dot{\boldsymbol{v}}_B \in \mathbb{R}^3$ ($\dot{\boldsymbol{\omega}}_B \in \mathbb{R}^3$). The rotation matrix ${}^W\boldsymbol{R}_B \in SO(3)$ and Euler angles $(\phi_B, \theta_B, \psi_B)$, corresponding to roll, pitch, and yaw of $\mathcal{F}_B$ in $\mathcal{F}_W$ are also used throughout the paper.

The OmniOcta introduced in our earlier work is an omnidirectional UAV equipped with eight propellers. The geometry of the propeller positions and orientations is optimized to achieve a balanced omnidirectional design, capable of generating a wrench $\boldsymbol{w} = [\boldsymbol{f}^\top \in \mathbb{R}^3, \boldsymbol{m}^\top \in \mathbb{R}^3]^\top \in \mathbb{R}^6$ at its CoM. Let $\boldsymbol{u} \in \mathbb{R}^8$ denote the control inputs, corresponding to the thrust produced by each propeller along its rotation axis. The UAV's equations of motion can then be written as

$$\begin{bmatrix} m_B\dot{\boldsymbol{v}}_B \\ \boldsymbol{J}_B\dot{\boldsymbol{\omega}}_B \end{bmatrix} = \begin{bmatrix} -m_B g\boldsymbol{e}_3 \\ \boldsymbol{\omega}_B \times \boldsymbol{J}_B\boldsymbol{\omega}_B \end{bmatrix} + \begin{bmatrix} {}^W\boldsymbol{R}_B & \boldsymbol{0}_{3\times3} \\ \boldsymbol{0}_{3\times3} & \boldsymbol{I}_3 \end{bmatrix} \boldsymbol{F}\boldsymbol{u}, \quad (1)$$

where $m_B \in \mathbb{R}_{>0}$ and $\boldsymbol{J}_B \in \mathbb{R}^{3\times3}_{>0}$ denote the mass and inertia matrix of the UAV, and $\boldsymbol{F} \in \mathbb{R}^{6\times8}$ is the allocation matrix mapping propeller thrusts to the applied wrench.

The UAV is designed such that $\text{rank}(\boldsymbol{F}) = 6$, allowing it to generate a 6D independent wrench components. Furthermore, it is capable of sustaining its own weight while maintaining arbitrary orientations.

B. Trajectory Modeling Requirements

The UAV operates in a cluttered environment populated with obstacles, with a state (velocity) [acceleration] in the configuration space defined as

$$\begin{aligned} \boldsymbol{\zeta} &= \begin{bmatrix} {}^W\boldsymbol{p}_B{}^\top & \boldsymbol{q}_B^\top \end{bmatrix}^\top \in \mathcal{C} \subseteq \mathbb{R}^3 \times \mathbb{H}, \\ \dot{\boldsymbol{\zeta}} &= \begin{bmatrix} {}^W\boldsymbol{v}_B{}^\top & \boldsymbol{\omega}_B^\top \end{bmatrix}^\top \in \mathcal{V} \subset \mathbb{R}^6, \\ \ddot{\boldsymbol{\zeta}} &= \begin{bmatrix} {}^W\dot{\boldsymbol{v}}_B{}^\top & \dot{\boldsymbol{\omega}}_B^\top \end{bmatrix}^\top \in \mathcal{A} \subset \mathbb{R}^6. \end{aligned}$$

The spaces $\mathcal{V}$ and $\mathcal{A}$ are bounded to ensure that any trajectory within $\mathcal{C}$ remains dynamically feasible. The full state of the UAV is therefore written as $\boldsymbol{\chi} = \begin{bmatrix} \boldsymbol{\zeta}^\top & \dot{\boldsymbol{\zeta}}^\top & \ddot{\boldsymbol{\zeta}}^\top \end{bmatrix}^\top \in \mathcal{X} \subset \mathbb{R}^3 \times \mathbb{H} \times \mathbb{R}^6 \times \mathbb{R}^6$.

The UAV trajectory $\boldsymbol{\chi}(t)$, is parameterized with respect to time $t \in [0, T]$, while the surrounding environment's point cloud is

$$\mathcal{P} = \{\boldsymbol{p}_i \in \mathbb{R}^3 \mid i = 1, 2, \ldots, N\},$$

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 03:53:39 UTC from IEEE Xplore. Restrictions apply.

and the UAV body is approximated by an ellipsoid $\mathcal{H}(\boldsymbol{\zeta})$. This ellipsoid has equal radii in the $\boldsymbol{x}_B$–$\boldsymbol{y}_B$ plane denoted as r_{xy} and a smaller radius along the $\boldsymbol{z}_B$ direction denoted r_z. We assume that the point cloud is updated online, reflecting newly discovered obstacles as well as dynamic changes in the environment.

The goal of the trajectory planning problem is to find a trajectory from a starting configuration $\boldsymbol{\zeta}_s \in \mathcal{C}$ to a goal configuration $\boldsymbol{\zeta}_g \in \mathcal{C}$, while avoiding the collision between the UAV and the discovered point cloud, *i.e.*, $\{\boldsymbol{p}_i \notin \mathcal{H}(\boldsymbol{\zeta}(t)) | \forall \boldsymbol{p}_i \in \mathcal{P}, \boldsymbol{\zeta}(t) \subset \boldsymbol{\chi}(t) \forall t \in [0, T]\}$.

The proposed trajectory planning and environment discovery is illustrated in Figure 2, where the trajectory planning approach can be summarized as follows:

1) An incremental sub-goal is computed in the discovered free space.
2) An initial 3D path is computed using A* from the current position of the drone to the sub-goal.
3) An optimal smooth 6D path is computed from the reference initial path, ensuring shortest length and obstacle avoidance.
4) A polynomial trajectory is fitted to the computed path.

III. SLAM and Optimal Path Computation

A. Simultaneous Localization and Mapping in SITL

Accurate localization and environment mapping are fundamental requirements for autonomous navigation and obstacle avoidance. In this work, the Real-Time Appearance-Based Mapping (RTAB-Map) framework [22] is employed for Simultaneous Localization and Mapping (SLAM) within both simulation and real-world experiments. RTAB-Map integrates visual odometry and loop-closure detection through an appearance-based bag-of-words approach, enabling long-term, graph-based SLAM.

The OmniOcta shown in Figure 1 is simulated in Gazebo using a virtual RGB-D (Intel RealSense D435i) camera model, which provides synchronized RGB, depth, and inertial measurements. These sensory inputs are processed by RTAB-Map to estimate the drone's pose and to incrementally construct a 3D point cloud map of the environment. The reconstructed scene is then represented through multiple spatial structures that support different stages of perception and planning:

- **Occupancy grid:** A 2D projection of the mapped environment used for subgoal selection within the regions already explored by the SLAM system.
- **OctoMap:** A 3D probabilistic, voxel-based representation that encodes free, occupied, and unknown regions, enabling volumetric reasoning and map integration.
- **Euclidean Distance Transform (EDT):** A continuous distance field that assigns each voxel its Euclidean distance to the nearest obstacle surface. The EDT is independently generated from the OctoMap and employed within the trajectory optimizer for collision-aware motion planning. The EDT is updated incrementally and thus does not require extensive processing power.

B. Optimal Path Computation

The computed path is assumed to consist of a series of splines, with three primary objectives: minimizing path length, ensuring spline smoothness, and preventing any spline from intersecting discovered obstacle points.

1) Path Representation: We model the path as a sequence of M spline segments. Each segment $f_k(s)$, $k = 1, \ldots, M$, is a degree-N spline:

$$f_k(s) = \sum_{i=0}^{N} p_{k,i}\, B_i(s-k), \qquad s \in [k, k+1], \tag{2}$$

where $B_i(s)$ are the chosen basis functions and $p_{k,i} \in \mathbb{R}$ are the segment coefficients (control points along one degree of freedom). We denote by $f(s)$, $s \in [0, M]$ the concatenation of all segments for one dimension.

Adjacent segments are joined with endpoint continuity:

$$p_{k,0} = p_{k-1,N}, \qquad k = 2, \ldots, M. \tag{3}$$

The coefficients are concatenated into a single vector $\mathbf{p}$ by taking all $N{+}1$ coefficients of the first segment and, for each subsequent segment, appending only the additional N coefficients:

$$\mathbf{p}^{(d)} = \big[\, p_{1,0},\, p_{1,1},\, \ldots,\, p_{1,N},\, p_{2,1},\, \ldots,\, p_{M,N} \,\big]^\top. \tag{4}$$

With this construction, the continuity constraints are enforced by definition (the shared endpoints are not duplicated). The total number of independent decision variables is therefore

$$n_{\text{var}} \;=\; (N{+}1) \;+\; (M{-}1)N \;=\; MN + 1. \tag{5}$$

a) Multiple Degrees of Freedom: For a D-dimensional path, a separate coefficient vector $\mathbf{p}^{(d)}$ is optimized for each dimension. The final decision variable vector is written as

$$\mathbf{P} = \big[\, \mathbf{p}^{(1)},\, \mathbf{p}^{(2)},\, \ldots,\, \mathbf{p}^{(D)} \,\big]^\top, \tag{6}$$

yielding a total of $D(MN + 1)$ decision variables.

b) Six Degrees of Freedom: In this work, we optimize a 6D trajectory consisting of the translational position $\big[x(s),\, y(s),\, z(s)\big]$ and the incremental orientation angles $\big[\Delta\phi(s),\, \Delta\theta(s),\, \Delta\psi(s)\big]$, where $\Delta\phi$, $\Delta\theta$, and $\Delta\psi$ denote the roll, pitch, and yaw increments with respect to the initial pose $\boldsymbol{\zeta}_s$. The decision vector therefore takes the form

$$\mathbf{p} = \big[\, x,\, y,\, z,\, \Delta\phi,\, \Delta\theta,\, \Delta\psi \,\big]^\top, \tag{7}$$

with each element parameterized by its own spline coefficient vector $\mathbf{p}^{(d)}$.

The quaternion is then recovered incrementally rather than from absolute Euler angles, in order to avoid singularities. At each sample j, we compute the incremental rotation quaternion

$$\delta q\big(\delta\phi(j), \delta\theta(j), \delta\psi(j)\big) \tag{8}$$

from the local roll–pitch–yaw differences

$$\begin{aligned} \delta\phi(j) &= \Delta\phi(j) - \Delta\phi(j-1), \\ \delta\theta(j) &= \Delta\theta(j) - \Delta\theta(j-1), \\ \delta\psi(j) &= \Delta\psi(j) - \Delta\psi(j-1) \end{aligned} \tag{9}$$

with respect to the previous sample, and update the orientation as

$$q(j) \;=\; q(j{-}1) \;\otimes\; \delta q\big(\delta\phi(j), \delta\theta(j), \delta\psi(j)\big), \tag{10}$$

starting from the known initial orientation $q(0) = q_s$, and where $\otimes$ refers to the Hamilton product between quaternions.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 03:53:39 UTC from IEEE Xplore. Restrictions apply.

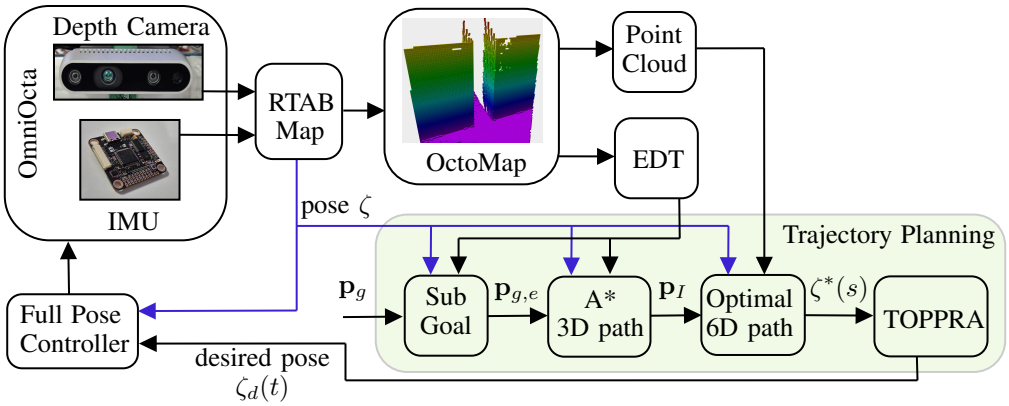


Fig. 2: Schematic of the proposed trajectory planning approach in cluttered and unknown environments for the OmniOcta drone.

2) Path Optimization Problem: Given the above formulation, the path optimization problem can be summarized as

$$\mathbf{P}^* = \arg\min_{\mathbf{P}} \; J_s + J_L + J_I, \qquad \text{s.t. } g_j \leq 0 \quad \forall j \in [0, N_s], \tag{11}$$

where J_s, J_L, and J_I represent respectively the smoothing cost, the length cost, and a cost that enforces similarity to an initial guess for the splines. The functions g_j are obstacle-avoidance constraints computed at N_s sampled points along the path. The consequent path is computed by sampling the spline functions, and is denoted as $\boldsymbol{\zeta}^*(s) \forall s \in [0, M]$ The different components of the cost and constraints are detailed below.

a) Path Cost Function: To encourage geometric smoothness independent of timing, we penalize spatial derivatives of the path. The employed smoothness cost is

$$J_\mathrm{s} = \int \left(w_t \left\| f'(s) \right\|^2 + w_\kappa \left\| f''(s) \right\|^2 + w_{\kappa'} \left\| f^{(3)}(s) \right\|^2 \right) ds, \tag{12}$$

where $f'(s)$ is the path tangent, $f''(s)$ the curvature vector, and $f^{(3)}(s)$ the curvature variation; $w_t, w_\kappa, w_{\kappa'} \geq 0$ are weights. Since spline derivatives are linear in the control points [15], the cost reduces to a quadratic form

$$J_\mathrm{s} = \mathbf{P}^\top \mathbf{Q}_1 \mathbf{P} + \mathbf{P}^\top \mathbf{Q}_2 \mathbf{P} + \mathbf{P}^\top \mathbf{Q}_3 \mathbf{P}, \tag{13}$$

with precomputed Gram matrices $\mathbf{Q}_k$ built from the spline basis derivatives with respect to s.

In addition to smoothness, the path length is penalized. The length cost for the position and orientation paths is defined as

$$J_\mathrm{L} = w_{L,p} \sum_{j=1}^{N_s} \|\boldsymbol{p}(j) - \boldsymbol{p}(j-1)\|^2 + w_{L,q} \sum_{j=1}^{N_s} \|[\delta\phi(j), \delta\theta(j), \delta\psi(j)]^\top\|^2, \tag{14}$$

where $w_{L,p}$ and $w_{L,q}$ represent respectively the translational and orientation path length weights.

b) Collision Avoidance: Collision avoidance is enforced against nearby obstacles, captured by the point cloud $\mathcal{P}$. The effective distance between the UAV and $\mathcal{P}$ is anisotropic due to the non-symmetrical shape of the drone. The UAV is modeled as an ellipsoid with radius r_{xy} in the $\boldsymbol{x}_B$–$\boldsymbol{y}_B$ plane and radius $r_z < r_{xy}$ in the $\boldsymbol{z}_B$ direction.

Let $\boldsymbol{R}(s)$ denote the rotation matrix corresponding to quaternion $\boldsymbol{q}_B(s)$. The anisotropic distance between the UAV at configuration $\boldsymbol{\zeta}(s)$ and the point cloud is defined as

$$d_{Ani}(\boldsymbol{\zeta}(s), \mathcal{P}) = \arg\min_{p_i \in \mathcal{P}} \operatorname{diag}\left(\tfrac{1}{r_{xy}}, \tfrac{1}{r_{xy}}, \tfrac{1}{r_z}\right) \boldsymbol{R}(s)^\top (\boldsymbol{p}(j) - \boldsymbol{p}_i). \tag{15}$$

The following constraint enforces collision avoidance

$$g_j = 1 - \|d_{Ani}(\boldsymbol{\zeta}(j), \mathcal{P})\|^2 \leq 0 \qquad \forall j \in [0, N_s]. \tag{16}$$

It is important to note that the initialization of the path strongly affects the feasibility of the final solution. If the initial guess places sampled points inside obstacles, or if a spline segment crosses an obstacle, the solver may fail to find a feasible path as it can become trapped in a local minimum.

C. Incremental Path Initialization

At the start of the mission, the map is empty, *i.e.*, all cells are initially assumed to be unknown. If a direct path to the global goal was planned under the assumption that unknown cells are free, it would likely fail to avoid undiscovered obstacles. To address this, an *incremental planning strategy* is adopted, where the UAV advances toward the global goal by following a sequence of local sub-goals. At each step, the planning process incorporates newly discovered obstacles, ensuring continuous adaptation.

1) Incremental Sub-goal: At iteration e, a local sub-goal $\boldsymbol{p}_{g,e}$ is chosen as the frontier point—a free location discovered by SLAM—that lies closest to the global goal. Since RTAB-Map does not directly provide frontier points for 3D maps, the global goal $\boldsymbol{p}_g$ is projected onto the 2D occupancy grid (see Fig. 4). A search is then applied to identify a free grid cell that satisfies the following conditions: 1) it is marked as free, 2) it is closest to the global goal, and 3) it is farthest from the current UAV position. The third condition helps avoid trivial local minima and promotes exploration.

The selected grid cell is then lifted to the altitude of the global goal to form a tentative sub-goal. Since this location may still be close to obstacles in 3D, we refine it using a depth-first search in the Euclidean Distance Transform (EDT) map to find the nearest point that is at least r_{xy} away from obstacles.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 03:53:39 UTC from IEEE Xplore. Restrictions apply.

2) Initial Guess via Graph Search: Once the local sub-goal is fixed, an A* search [23] computes a feasible path from the UAV's current position $\boldsymbol{p}_s$ to $\boldsymbol{p}_{g,e}$ within the EDT map, ensuring clearance of at least r_{xy}. The search in the EDT map ensures a fast computation of the A* path, as distance to obstacles are precomputed. A spline is then fitted to this path, and its coefficients serve as the initial guess $\mathbf{P}_I$ for the trajectory optimization. For orientations, the initialization is obtained by linearly interpolating between the current orientation and a unit quaternion aligned with the sub-goal direction.

3) Initialization Cost: To keep the optimized solution close to the initialization, we introduce the cost term:

$$J_I = w_I \sum_{i=1}^{3} \|\mathbf{p}^{(i)} - \mathbf{p}_I^{(i)}\|^2, \tag{17}$$

where $\mathbf{p}^{(i)}$ and $\mathbf{p}_I^{(i)}$ denote the coefficient vectors of the optimized and initial spline in the i-th spatial dimension, respectively.

IV. Trajectory Generation & Controller Design

A. Time-Optimal Trajectory

Once a collision-free path has been computed, the next step is to generate a time-parameterized trajectory that respects dynamic constraints while minimizing traversal time. At each sampled path point k, we evaluate the minimum distance to obstacles $d_{\min,k}$ and compute a corresponding velocity bound $\overline{\boldsymbol{v}}_k$ similar to our previous work [21].

To obtain the time-optimal trajectory, we employ the `toppra` library [24], which implements a convex optimization-based approach for time-parameterization of paths under velocity and acceleration bounds. Given the geometric path, `toppra` computes Bézier representations of $\boldsymbol{\zeta}(t)$, $\dot{\boldsymbol{\zeta}}(t)$, and $\ddot{\boldsymbol{\zeta}}(t)$ such that the traversal time

$$T = \min \sum_{k=0}^{K-1} \int_{\boldsymbol{s}_k} \frac{ds}{\dot{\boldsymbol{\zeta}}(\boldsymbol{s}_k)} \tag{18}$$

is minimized subject to

$$-\overline{\boldsymbol{v}}_k \le \dot{\boldsymbol{\zeta}}(\boldsymbol{s}_k) \le \overline{\boldsymbol{v}}_k, \tag{19}$$

$$-\overline{\ddot{\boldsymbol{\zeta}}} \le \ddot{\boldsymbol{\zeta}} \le \overline{\ddot{\boldsymbol{\zeta}}}, \tag{20}$$

where $\boldsymbol{s}_k$ denotes the path segment between points k and $k+1$. The key advantage of `toppra` is its ability to handle spatially varying velocity limits, which we leverage to impose tighter bounds near obstacles (small $d_{\min,k}$) while allowing higher speeds in open regions similar to our previous work [20], [21]. Acceleration bounds remain constant throughout.

B. Controller Design

The time-parameterized trajectory provides smooth references for position, orientation, and their derivatives over the entire interval $t \in [0, T]$. The role of the controller is to track these references with high accuracy by generating appropriate force and torque commands.

a) Desired Wrench.: Let $[\cdot]^d$ denote desired quantities. We define the desired wrench $\boldsymbol{w}^d = [{}^W\boldsymbol{f}^{d,\top}, \boldsymbol{m}^{d,\top}]^\top$ as

$$\begin{aligned}
{}^W\boldsymbol{f}^d &= g m_B \boldsymbol{z}_W + \dot{\boldsymbol{v}}_B^d + \boldsymbol{K}_p \boldsymbol{e}_p + \boldsymbol{K}_v \boldsymbol{e}_v + \boldsymbol{K}_{i,p} \int_0^t \boldsymbol{e}_p \, d\tau, \\
\boldsymbol{m}^d &= \boldsymbol{\omega}_B \times \boldsymbol{J}_B \boldsymbol{\omega}_B + \boldsymbol{K}_R \boldsymbol{e}_R + \boldsymbol{K}_w \boldsymbol{e}_w + \boldsymbol{K}_{i,R} \int_0^t \boldsymbol{e}_R \, d\tau, \\
\boldsymbol{w}^d &= \begin{bmatrix} {}^W\boldsymbol{R}_B & \mathbf{0}_{3\times3} \\ \mathbf{0}_{3\times3} & \boldsymbol{I}_3 \end{bmatrix} \begin{bmatrix} {}^W\boldsymbol{f}^d \\ \boldsymbol{m}^d \end{bmatrix},
\end{aligned} \tag{21}$$

where the errors are defined as

$$\boldsymbol{e}_p = \boldsymbol{p}_B^d - \boldsymbol{p}_B, \quad \boldsymbol{e}_v = \boldsymbol{v}_B^d - \boldsymbol{v}_B, \quad \boldsymbol{e}_w = \boldsymbol{\omega}_B^d - \boldsymbol{\omega}_B,$$

and the rotation error is

$$\boldsymbol{e}_R = \frac{1}{2}\big({}^W\boldsymbol{R}_B^{d\top}\,{}^W\boldsymbol{R}_B - {}^W\boldsymbol{R}_B^\top\,{}^W\boldsymbol{R}_B^d\big)^\wedge.$$

The matrices $\boldsymbol{K}_p, \boldsymbol{K}_v, \boldsymbol{K}_{i,p}, \boldsymbol{K}_R, \boldsymbol{K}_w, \boldsymbol{K}_{i,R} \in \mathbb{R}^{3\times3}$ are diagonal and contain tunable control gains.

b) Control Allocation.: Given the desired wrench $\boldsymbol{w}^d$, the corresponding thrust commands are obtained through control allocation:

$$\begin{aligned}
\boldsymbol{u}^d = \boldsymbol{u}_c + \boldsymbol{B} \arg\min_{\boldsymbol{\lambda}\in\mathbb{R}^2} \|\boldsymbol{u}_c + \boldsymbol{B}\boldsymbol{\lambda}\|, \\
\text{s.t.} \quad \boldsymbol{u}_c = \boldsymbol{F}^\dagger \boldsymbol{w}^d, \;\; \boldsymbol{u}^d \in \mathcal{U}
\end{aligned} \tag{22}$$

where $\boldsymbol{F}^\dagger$ is the Moore–Penrose pseudoinverse of the actuation matrix $\boldsymbol{F}$, $\boldsymbol{B}$ spans the null space of $\boldsymbol{F}$, $\boldsymbol{B}\boldsymbol{\lambda}$ represents feasible adjustments that preserve the achieved wrench while satisfying actuator constraints, and $\mathcal{U}$ is the set of feasible controls.

V. SITL-based imulation Studies

A. Simulation Setup

The validation is carried out using the *Gazebo* Physics Engine simulation environment that reproduces a structured obstacle course. The geometric layout is retained from our previous work [20] while the walls and obstacles are enhanced with textured patterns to support visual odometry (VO) and feature-based localization. The textured environment provides sufficient visual contrast and gradient variation for reliable keypoint extraction and matching, thereby improving the accuracy of motion estimation during omnidirectional maneuvers. The workspace spans 20 m × 20 m with a height of 5 m. At the start of the simulation, the origin of the world frame O_W is aligned with the origin of the UAV body frame O_B, and their orientations are identical. The vehicle must pass through two narrow openings and around a single box-shaped obstacle. Each opening is slightly larger than twice the vehicle's minimum radius $r_z = 0.35\,m$ and smaller than twice the vehicles's larger radius $r_{xy} = 0.6\,m$, necessitating tilted orientations for safe traversal-maneuvers that cannot be operated by underactuated multirotors at low translation speeds. The resulting obstacle map used for both planning and perception is illustrated in Figure 3.

B. Path Computation

The proposed planner generates the global trajectory through an iterative process alternating between sub-goal selection, path generation, and trajectory optimization. At each iteration, a new sub-goal is selected from within the currently explored portion of the map, as maintained by the

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 03:53:39 UTC from IEEE Xplore. Restrictions apply.

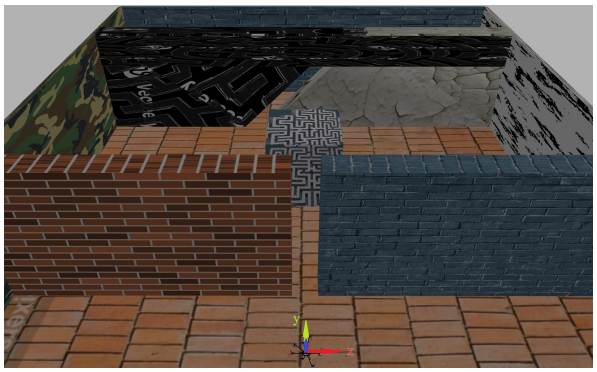


Fig. 3: Textured map environment [20] used for simulation and VO.

SLAM system. The selection operates on the 2D occupancy grid shown in Figure 4, which encodes free (white), occupied (black), and undiscovered (gray) regions.

Figure 4 illustrates the occupancy grid used during the planning iteration. Starting from sub-goal 1, the UAV evaluates potential candidates within the discovered area, considering both its current pose and the global goal location. The selected sub-goal 2 lies along a region that advances the vehicle toward the final goal while remaining inside the safely mapped boundary. This choice is determined through a weighted criterion that jointly favors positions closer to the global goal and farthest from the current pose, thereby encouraging safe progress without entering unknown or risky areas, and while exploring the environment.

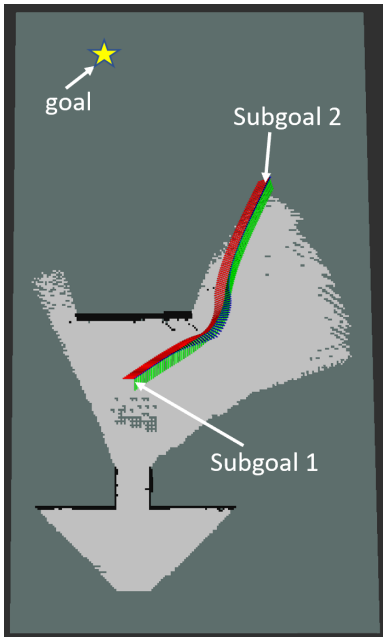


Fig. 4: Occupancy grid used for sub-goal selection.

Once a sub-goal is determined, a collision-free A* path is computed between the current pose and the selected sub-goal. The resulting path is then refined through the 6D trajectory optimization stage that enforces smoothness, dynamic feasibility, and attitude alignment. As the UAV follows the optimized trajectory, both the occupancy grid and OctoMap, shown in Figure 5, are incrementally updated, triggering the selection of subsequent sub-goals based on the newly discovered regions.

Complementary to the 2D grid, the 3D-OctoMap of the accumulated point cloud is maintained throughout the mission. From this map we compute an Euclidean Distance Transform (EDT), which provides fast clearance queries to the nearest obstacle. The generated OctoMap for the implemented scenario is shown in Fig. 5 at the end of the trajectory, where the colored voxels encode the occupancy probability of each region.

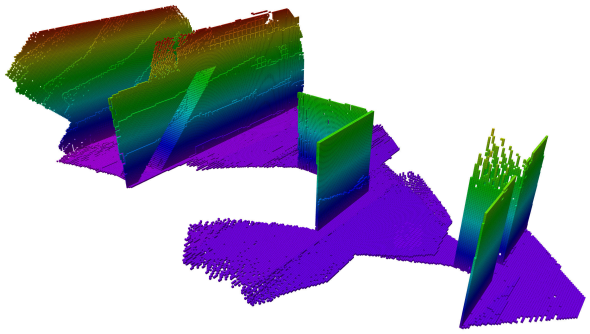


Fig. 5: 3D-OctoMap representation of the environment.

Table I lists the initial pose, all intermediate sub-goals, and the final goal used in this scenario. Figures 6 and 7 present the overall outcome of this iterative process, showing the optimized path, intermediate A* paths, sub-goals, and the cumulative map of the explored environment. Colored arrows indicate the UAV body orientation along the path. In several areas, the optimized trajectory deviates from the grid-based A* path, as the optimizer eliminates unnecessary detours and enforces dynamically smooth transitions. These refinements result in a spatially efficient and feasible motion while preserving proper attitude alignment through confined gaps.

TABLE I: Initial pose, subgoals, and final goal positions.

Waypoint	x [m]	y [m]	z [m]
Initial pose	0.00	0.00	1.50
Subgoal 1	0.05	6.00	1.50
Subgoal 2	3.60	10.40	1.50
Subgoal 3	-0.45	12.60	1.50
Final goal	-1.50	17.00	1.50

At the first vertical gap, the UAV performs an approximately 90° pitch rotation to align with the opening. Around the central obstacle, two subgoals are used to efficiently negotiate the corner given the limited map knowledge available at that stage. Finally, near the goal, the drone reorients itself towards the tilted gap to ensure safe passage. The UAV's orientation along the path is visualized using body-frame arrows where red, green, and blue correspond to the $\boldsymbol{x}_B$, $\boldsymbol{y}_B$, and $\boldsymbol{z}_B$ axes, respectively.

C. Safe Trajectory Planning

After computing the paths between sub-goals, the `toppra` library described in Section IV-A is employed to generate dynamically feasible trajectories and assess their safety. The velocity and acceleration bounds are selected to promote smooth maneuvering while preserving accurate state estimation and preventing map drift. The linear velocity

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 03:53:39 UTC from IEEE Xplore. Restrictions apply.

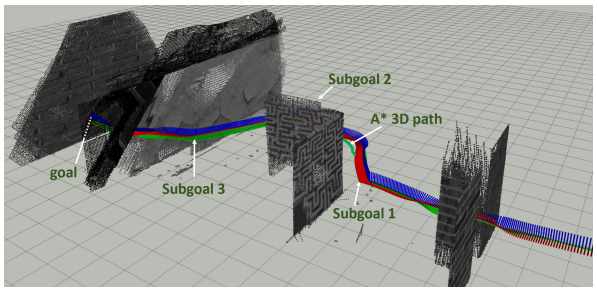


Fig. 6: Isometric view of the iterative planning process showing the optimized trajectory, A* paths, and sub-goals.

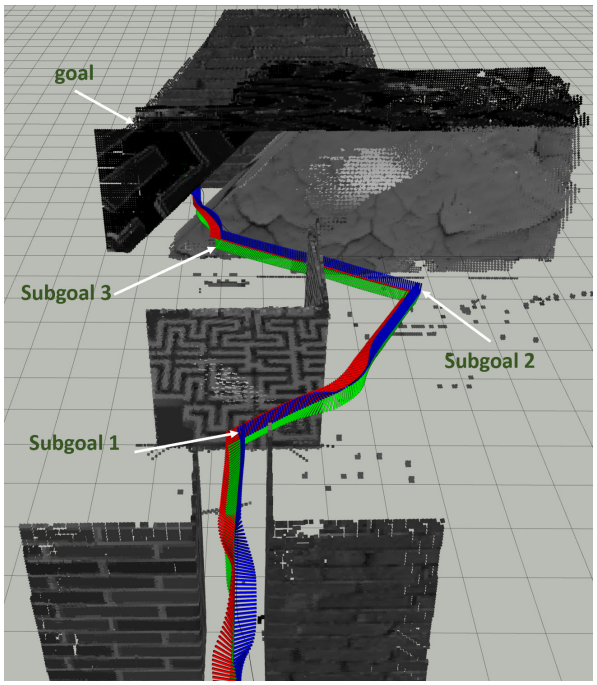


Fig. 7: Top-view showing the sub-goals and the accumulated point cloud of the discovered regions.

limits are set to $[1,\ 1,\ 1]$ m/s with corresponding linear acceleration limits of $[0.5,\ 0.5,\ 0.5]$ m/s^2. Angular velocity and acceleration are constrained to $[0.3,\ 0.3,\ 0.1]$ rad/s and $[0.1,\ 0.1,\ 0.1]$ rad/s^2, respectively. To further enhance safety, the allowable linear and angular velocities are modulated according to the distance to the nearest forward obstacle, following the adaptive strategy in [20]. This mechanism reduces speed in cluttered regions and permits higher velocities in open areas. Additionally, the pitch and roll angular velocity bounds are chosen higher than those of yaw to mitigate map drift resulting from rapid yaw rotations or low-frequency oscillations in pitch and roll during slow turns.

Figure 8 presents the resulting 6D trajectory tracking performance, comparing the desired and actual states over time. The upper plot shows the position tracking in x, y, and z, while the lower part depicts the Euler angles (roll, pitch, yaw). The UAV exhibits precise tracking and smooth attitude transitions during each maneuver where the colored regions represent the pose hold intervals. The UAV closely follows the desired trajectory with smooth transitions between sub-goals and minimal overshoot, demonstrating that the time-optimal trajectories remain dynamically feasible under the imposed constraints. This confirms that the proposed planner successfully balances time efficiency with safety, yielding aggressive yet stable flight through cluttered environments.

In the same Figure 8, short pose holding intervals are visible before each major maneuver. These intervals represent the time during which the drone computed the next trajectory segment. The durations of these hold phases are approximately $[5.4,\ 5.7,\ 6.5,\ 6.2]$ s. The gradual increase in computation time across the first three maneuvers corresponds to the expanding size of the mapped environment, as each subsequent segment requires processing a larger portion of the accumulated OctoMap. However, the final hold interval deviates from this trend since the UAV was already near the final gap, resulting in a shorter computation phase.

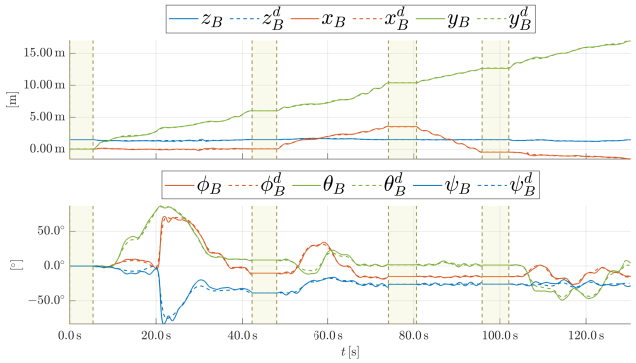


Fig. 8: Simulation results showing the desired vs. actual positions (top) and Euler angles (bottom).

VI. CONCLUSIONS

This paper presented a trajectory planning framework for omnidirectional UAVs operating in GPS-denied, cluttered environments. The approach integrates real-time perception with local and global planning by continuously generating feasible sub-goals from the explored free space and computing full-pose trajectories that jointly consider position and orientation. Through this formulation, the UAV maintains safe navigation in complex environments while fully exploiting its actuation capabilities to align its orientation with narrow or tilted openings that are otherwise infeasible for conventional underactuated drones.

The framework was validated in a physics-based simulation environment that replicates real-world visual and dynamic conditions. The results demonstrated the UAV's ability to traverse confined passages, adapt its orientation in response to obstacle geometry, and maintain stability during complex maneuvers. These findings confirm the planner's effectiveness in achieving both safety and efficiency under realistic perception and control constraints. Future work will focus on deploying the proposed framework on the physical omnidirectional platform to evaluate its performance in real-world, GPS-denied environments.

REFERENCES

[1] K. Tong, Y. Hu, B. Dikic, S. Solmaz, F. Fraundorfer, and D. Watzenig, "Robots saving lives: A literature review about search and rescue (SaR) in harsh environments," in *2024 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2024, pp. 953–960.

[2] D. Chaikalis, F. Khorrami, and A. Tzes, "Adaptive control approaches for an unmanned aerial manipulation system," in *2020 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE, 2020, pp. 498–503.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 03:53:39 UTC from IEEE Xplore. Restrictions apply.

[3] K. Ebadi, L. Bernreiter, H. Biggie, G. Catt, Y. Chang, A. Chatterjee, C. E. Denniston, S.-P. Deschênes, K. Harlow, S. Khattak *et al.*, "Present and future of SLAM in extreme environments: The DARPA Sub T challenge," *IEEE Transactions on Robotics*, vol. 40, pp. 936–959, 2023.
[4] M. Dharmadhikari, N. Khedekar, P. De Petris, M. Kulkarni, M. Nissov, and K. Alexis, "Maritime vessel tank inspection using aerial robots: Experience from the field and dataset release," *arXiv preprint arXiv:2404.19045*, 2024.
[5] A. Ollero, M. Tognon, A. Suarez, D. Lee, and A. Franchi, "Past, Present, and Future of Aerial Robotic Manipulators," *IEEE Transactions on Robotics*, vol. 38, no. 1, pp. 626–645, 2021.
[6] R. Rashad, J. Goerres, R. Aarts, J. B. Engelen, and S. Stramigioli, "Fully actuated multirotor UAVs: A literature review," *IEEE Robotics & Automation Magazine*, vol. 27, no. 3, pp. 97–107, 2020.
[7] M. Hamandi, Q. Sable, M. Tognon, and A. Franchi, "Understanding the Omnidirectional Capability of a Generic Multi-rotor Aerial Vehicle," in *2021 Aerial Robotic Systems Physically Interacting with the Environment (AIRPHARO)*, 2021, pp. 1–6.
[8] A. Ali, C. Gabellieri, and A. Franchi, "A control theoretic study on omnidirectional mavs with minimum number of actuators and no internal forces at any orientation," in *2024 IEEE 63rd Conference on Decision and Control (CDC)*, 2024, pp. 3571–3576.
[9] ——, "Exploring a new design paradigm for omnidirectional mavs for minimal actuation and internal force elimination: Theoretical framework and control," 2025. [Online]. Available: https://arxiv.org/abs/2510.15071
[10] T. Hui, E. Cuniato, M. Pantic, M. Tognon, M. Fumagalli, and R. Siegwart, "Passive aligning physical interaction of fully-actuated aerial vehicles for pushing tasks," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 6152–6158.
[11] M. Hamandi, A. M. Ali, K. Kyriakopoulos, A. Tzes, and F. Khorrami, "An Omnidirectional Non-Tethered Aerial Prototype with Fixed Unidirectional Thrusters," in *2025 International Conference on Robotics and Automation (ICRA)*. Atlanta, GA, USA: IEEE, May 2025, pp. 8649–8655.
[12] M. Hamandi, A. M. Ali, N. Evangeliou, D. Chaikalis, A. Tzes, K. Kyriakopoulos, and F. Khorrami, "Mechatronic Design of an Omnidirectional Octorotor UAV," in *2024 10th International Conference on Automation, Robotics and Applications (ICARA)*, 2024, pp. 300–304.
[13] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "Ego-planner: An esdf-free gradient-based local planner for quadrotors," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 478–485, 2021.
[14] L. Quan, Z. Zhang, X. Zhong, C. Xu, and F. Gao, "Eva-planner: Environmental adaptive quadrotor planning," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 398–404.
[15] C. Richter, A. Bry, and N. Roy, "Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments," in *Robotics Research*, 2016, p. 649–666.
[16] D. Brescianini and R. D'Andrea, "Computationally Efficient Trajectory Generation for Fully Actuated Multirotor Vehicles," *IEEE Transactions on Robotics*, vol. 34, no. 3, pp. 555–571, 2018.
[17] F. Morbidi, D. Bicego, M. Ryll, and A. Franchi, "Energy-Efficient Trajectory Generation for a Hexarotor with Dual-Tilting Propellers," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 6226–6232.
[18] M. Pantic, L. Ott, C. Cadena, R. Siegwart, and J. Nieto, "Mesh Manifold based Riemannian Motion Planning for Omnidirectional Micro Aerial Vehicles," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4790–4797, 2021.
[19] P. Liu, F. Quan, Y. Liu, and H. Chen, "Collision-free 6-DoF Trajectory Generation for Omnidirectional Multi-rotor Aerial Vehicle," *arXiv preprint arXiv:2209.06764*, 2022.
[20] A. M. Ali, M. Hamandi, and A. Tzes, "Efficient safe trajectory planning for an omnidirectional drone," in *2025 International Conference on Unmanned Aircraft Systems (ICUAS)*, 2025, pp. 785–792.
[21] M. Hamandi, A. M. Ali, A. Tzes, and F. Khorrami, "Experimental Evaluation of Safe Trajectory Planning for an Omnidirectional UAV," in *2025 International International Conference on Intelligent Robots and Systems(IROS)*. Hangzhou, CHINA: IEEE, Oct. 2025, (accepted).
[22] M. Labbé and F. Michaud, "Rtab-map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation," *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.
[23] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
[24] H. Pham and Q. C. Pham, "A New Approach to Time-Optimal Path Parameterization Based on Reachability Analysis," *IEEE Transactions on Robotics*, vol. 34, pp. 645 – 659, 06 2018.

Authorized licensed use limited to: Korea Advanced Inst of Science & Tech - KAIST. Downloaded on November 02,2026 at 03:53:39 UTC from IEEE Xplore. Restrictions apply.